**Practical No.1**

**Aim: Implementing Substitution and Transposition Ciphers: Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.**

---

**Code for implementing Substitution Cipher with Caesar Cipher:**

```
def caesar_cipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            offset = ord('a') if char.islower() else ord('A')
            result += chr((ord(char) - offset + shift) % 26 + offset)
        else:
            result += char
    return result


# Example usage
message = input("Enter the text to encrypt: ")
shift = 3

encrypted_message = caesar_cipher(message, shift)
print("Encrypted:", encrypted_message)

decrypted_message = caesar_cipher(encrypted_message, -shift)
print("Decrypted:", decrypted_message)
```

**Output :**

```
======================== RESTART: E:\INS\Ceasercipher.py
Enter the text to encrypt: PRIYA RAJAK
Encrypted: SULBD UDMDN
Decrypted: PRIYA RAJAK
```

SFCM 2025-26

**Code for implementing transposition Cipher using Railfence Cipher :**

```
def rail_fence_encrypt(text):
    return text[::2] + text[1::2]


def rail_fence_decrypt(cipher):
    half = (len(cipher) + 1) // 2
    first, second = cipher[:half], cipher[half:]
    return ''.join(
        first[i // 2] if i % 2 == 0 else second[i // 2]
        for i in range(len(cipher))
    )


# Main
if __name__ == "__main__":
    text = input("Enter the text: ")

    encrypted = rail_fence_encrypt(text)
    print("Encrypted:", encrypted)

    decrypted = rail_fence_decrypt(encrypted)
    print("Decrypted:", decrypted)
```

**Output:**

```
======================= RESTART: E:\INS\Raifencecipher.py =
Enter the text: PRIYARAJAK
Encrypted: PIAAARYRJK
Decrypted: PRIYARAJAK
```

SDSIT 2025-26